

M1 – MASTER SAR
Premier examen réparti

Novembre 2016

2 heures – Tout document papier autorisé

Barème donné à titre indicatif

L. Arantes, P. Cadinot, E. Encrenaz, J. Lejeune, P. Sens, J. Sopena

I. SYNCHRONISATION (6 POINTS)

Nous considérons que des fichiers peuvent être accédés concurremment à condition que ces accès concernent le même type d'opération. Pour simplifier, nous ne considérons que deux types d'opérations: *OP_0* et *OP_1*. Ce que font ces opérations n'est pas important pour l'exercice.

Nous voulons alors offrir aux programmeurs les appels systèmes suivants :

- *void flock_op (int fd, int op)* : permet au processus appelant de demander l'accès à un fichier, depuis le descripteur *fd*, afin d'exécuter l'opération *op* sur ce fichier.
- *frelease_op(int fd)* : permet au processus appelant de libérer l'accès au fichier depuis le descripteur *fd*.

Exemple d'appel:

```
#define OP_0 0
#define OP_1 1

int fd;

main (int argc, char** argv) {
    ....
    fd=open("file1", O_RDONLY);
    flock_op (fd, OP_1)
    /* accès au fichier file1 pour exécuter OP_1*/
    .....
    frelease_op(fd);
    ....
}
```

Nous considérons que les processus utilisent correctement les appels systèmes.

Un processus qui appelle *flock_op (fd, i)* peut accéder au fichier correspondant à *fd* si aucun accès est en cours pour ce fichier ou si les accès en cours concernent l'opération *i*. Sinon, l'opération courante est *j* ($j = (i+1)\%2$) et le processus appelant doit attendre qu'il n'y ait plus d'accès pour l'opération *j* à ce fichier.

Autrement dit, lorsque tous les processus qui accédaient au fichier pour l'opération *j* ont libéré leur accès (appel à *frelease_op(fd)*), les processus demandeurs pour l'opération *i* peuvent accéder au fichier concurremment. Par conséquent, les nouveaux demandeurs pour l'opération *j* doivent être bloqués.

Pour implémenter les fonctions du noyau `sys_flock_op ( )` et `sys_frelease ( )` correspondantes aux deux appels système décrits ci-dessus, nous utiliserons l'inode du fichier. Vous pouvez ajouter d'autres champs à la structure `struct inode` ainsi qu'utiliser les drapeaux `ILOCK` et `IWANT` et en définir d'autres, si nécessaire. La priorité de réveil des processus endormis sera `PINOD`.

```
struct inode {
    char i_flag;
    ...
}
```

I.1. (4 points)

Donnez les pseudo-codes des fonctions du noyau `sys_flock_op ( )` et `sys_frelease ( )`. Indiquez sur quelle adresse les processus s'endorment et les champs ajoutés à la structure `struct inode` ainsi que les drapeaux.

Note: Dans ces fonctions, l'inode du fichier peut être accédé de la façon suivante:

```
struct inode * ip = u.u_ofile[u.arg[0]] ->f_inode;
```

```
#define OP_0 0
#define OP_1 1
#define AUCUNE -1

struct inode {
    ....
    int cur_op; /* opération courante -1, 0 ou 1 */
    int nb_op[2]; /* nombre d'opération en attente pour l'opération i */
}
```

Si aucune opération en cours, `cur_op = -1 (AUCUNE)`

Note : Les processus qui demandent l'opération `i` se bloquent sur l'adresse `&(ip->nb_op[i])`. Ils auraient pu s'endormir, indépendante de l'opération, sur la même adresse (e.g. adresse de l'inode). Mais la question I.2 nécessite de deux adresses différentes.

```
void sys_flock_op( ) {
    struct inode *ip = u.u_ofile[u.arg[0]] ->f_inode;
    int i = u.arg[1]; /* opération */
    ip->nb_op[i]++;
    if (ip->cur_op == (i+1)%2 )
        sleep(&(ip->nb_op[i]),PINOD);

    else
        if (ip->cur_op== AUCUNE)
            ip->cur_op = i;
```

```

}

void sys_release_op( ) {
    struct inode *ip = u.u_ofile[u.arg[0]] ->f_inode;
    if (--(ip->nb_op[ip->cur_op]) == 0)
        if (ip->nb_op[(ip->cur_op+1)%2] > 0) {
            /* donner la main à l'autre opération */
            ip->cur_op = (ip->cur_op+1)%2;
            wakeup(&(ip->nb_op[ip->cur_op]));
        }
        else
            ip->cur_op=AUCUNE;
}

```

La solution précédente présente un problème de famine. Par exemple, si des processus demandent sans arrêt à exécuter l'opération *OP_1* sur un fichier, il se peut que d'autres processus demandeurs pour l'opération *OP_0* sur ce même fichier restent bloqués pour toujours.

Pour résoudre ce problème, nous allons borner le temps d'accès concurrent d'une même opération sur un fichier en utilisant les fonctions *callout* de la façon suivante:

Lorsque le premier accès à l'opération *i* est autorisé un *timer (callout)* de *OP_TICKS* tiques doit être démarré. A l'expiration de ce *callout*, les nouvelles demandes pour l'opération *i* doivent être bloquées. Lorsqu'il n'y aura plus aucun accès en cours pour l'opération *i*, les processus demandeurs de l'opération *j* doivent être libérés et accèdent alors au fichier. Cependant, s'il n'y a pas de demandeurs pour l'opération *j*, les nouveaux demandeurs pour l'opération *i*, s'il y en a, peuvent accéder au fichier.

Pour cette solution vous devez utiliser les fonctions *timeout* et *untimeout* vues en TME:

```

/* renvoie un id. qui identifie le timeout */
int timeout (void (*fun) ( ), void *arg, int tim)

/* annule le timeout identifié par id. */
int untimeout (int id)

```

Si vous en avez besoin, vous pouvez aussi utiliser le drapeau *ITIMEOUT*

I.2. (2 points)

Donnez les nouveaux pseudo-codes des fonctions *sys_flock_op ()* et *sys_frelase ()* et le pseudo_code de la fonction appelée lors de l'expiration du *timeout*. Indiquez sur quelle adresse les processus s'endorment et les champs que vous avez ajoutés à la structure *struct inode*.

```

struct inode {
    ....
    int cur_op; /* opération courante -1, 0 ou 1 */
}

```

```

    int nb_op[2]; /* nombre d'opération pour l'opération i */
    int id_timeout; : * para o id do timeout */
}
/* fonction à être exécuter lors d'expiration du timeout */
fin_op (struct node* ip) {
    ip->i_flag |= ITIMEOUT;
}

void sys_flock_op( ) {
    struct inode *ip = u.u_ofile[u.arg[0]] ->f_inode;
    int i = u.arg[1]; /* opération */
    ip->nb_op[i]++;
    if ( (ip-> cor_op == (i+1)%2) || (ip ->i_flag & ITIMEOUT))
        sleep(&(ip->nb_op[i]),PINOD);
    }
    else
        if (ip->cor_op == AUCUNE)
            ip-> cor_op = i;

    if (ip->nb_op[i] == 1)
        /* premier demandeur */
        timeout(fin_op, ip, OP_TICKS);
}

void sys_release_op( ) {
    struct inode * ip = u.u_ofile[u.arg[0]] ->f_inode;
    if ( --(ip->nb_op[ip->cur_op]) == 0) {
        if (! (ip->i_flag & ITIMEOUT) )
            untimeout(ip->id_timeout);
        ip->i_flag &= ~TIMEOUT;
        if (ip->nb_op[(ip->cur_op+1)%2] >0)
            /* donner la main à l'autre opération */
            ip->cur_op = (ip->cur_op+1)%2;

        if (ip->nb_op[ip->cur_op] >0 ) {
            timeout(fin_op, ip, OP_TICKS);
            wakeup(&(ip->nb_op[ip->cur_op]));
        }
        else
            ip->cur_op=AUCUNE;
    }
}

```

```
}
```

II. APPEL SYSTEME ADOPT_GRANDCHLD () (6 POINTS)

Nous voulons offrir l'appel système *int adopt_grandchld ()* qui permet d'adopter les fils des fils du processus. Autrement dit, les petit-fils du processus appelant deviennent ses fils directs en cessant d'être les fils de leur père d'origine. L'appel doit rendre le nombre de fils adoptés par le processus appelant.

Dans un premier temps, nous considérons que lors de l'appel à *adopt_grandchld ()*, aucun des processus pères d'origine étaient en train d'attendre la terminaison d'un des leurs "ex-fils".

II.1. (3,5 points)

Donnez le pseudo-code de la fonction du noyau *int sys_adopt_grandchld ()* appelée par l'appel système *int nb_adopt_grandchld()*.

```
int sys_adopt_grand_chld ( ) {
    int n=0;
    struct proc* p,q;

    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_ppid == u.u_procp->p_pid )
            for (q = &proc[0]; q < &proc[NPROC]; q++)
                if (q->p_ppid == p->p_pid ) {
                    n++;
                    q->p_ppid = u.u_procp ->p_pid;
                }

    u.u_ar0[R0]=n;
    return;
}
```

Nous considérons maintenant que lors de l'appel à *int sys_adopt_grandchld ()*, il peut y avoir des processus fils de l'appelant qui attendait la terminaison d'un des leurs "ex-fils".

II.2. (1,5 points)

Modifiez en conséquence la fonction *sys_adopt_grandchld*. Votre fonction ne doit pas essayer de réveiller un processus fils qui ne se trouve pas bloqué en attente d'un de leur "ex-fils".

```
int sys_adopt_grandchld ( ) {
    int n=0;
    struct proc* p,q;

    for (p = &proc [ 0 ]; p < &proc [NPROC]; p++)
        if (p->p_ppid == u.u_procp->p_pid ) {
            for (q = &proc[0]; q < &proc[NPROC]; q++)
                if (q->p_ppid == p->p_pid ) {
                    if (p->w_chan == p)
                        wakeup (p);
                    n++;
                    q->p_ppid=u.u_procp->p_pid;
                }
        }
}
```

```
    }  
  
    u.u_ar0[R0]=n;  
    return;  
}
```

Le code de la fonction noyau `wait( )` appelée par l'appel système `int wait (int* status)` est donné en annexe.

II.3. (1 point)

Sans recopier la fonction `wait( )`, quelle modification on doit on apporter à celle-ci en considérant que le père du processus appelant le `wait (int* status)` peut appeler `sys_adopt_grandchild ( )` lorsque le processus attend la terminaison d'un de ses fils ?

Il faut juste déplacer le label **loop** de la ligne 7 vers la ligne 5 pour initialiser `f=0`.

III. SIGNAUX : APPEL SYSTEME NB_SIGPEND () (8 POINTS)

Nous voulons offrir l'appel système *int nb_sigpend ()* qui renvoie le nombre de signaux pendants du processus appelant.

III.1. (3,5 points)

Donnez le pseudo-code de la fonction du noyau *int sys_nb_sigpend ()* appelée par l'appel système *int nb_sigpend ()*.

```
int sys_nb_sigpend() {  
  
    int nb=u.uarg[0];  
    int cpt; long n;  
  
    struct proc *p= u.u_procp;  
  
    cpt =0 ;  
    n = p->p_sig;  
  
    for (i=1; i<NSIG ; i++){  
        if (n & 1L){  
            cpt++;  
            n>>1;  
        }  
    }  
  
    u.uar0[R0]=n;  
    return;  
}
```

Nous changeons maintenant le prototype de la fonction en y ajoutant le paramètre *int nb* : *int sys_nb_sigpend (int nb)*. Comme dans la question précédente, la nouvelle fonction renvoie le nombre de signaux pendants du processus appelant. Cependant, ce nombre doit être supérieur ou égale à la valeur *nb*, passée en paramètre. Autrement dit, tant que le nombre de signaux pendants est inférieur à *nb*, le processus appelant doit s'endormir. La fonction ne se termine que lorsque *nb* ou plus signaux se trouvent pendants pour ce processus.

Note : Nous considérons pour cette question que, si le processus, qui se trouve dans la fonction *int nb_sigpend (int n)*, est réveillé dû à un signal, la fonction *sleep* n'appellera pas *issig ()* pour tester s'il y a des signaux pendants, ce qui entrainerait la terminaison de la fonction avec le code de renvoi -1 (voir code de la fonction *sleep*).

III.2. (3 points)

Modifiez le code de la fonction *int sys_nb_sigpend ()* de la question précédente en conséquence en indiquant sur quelle adresse le processus s'endormira. Dans ce cas, il s'endort avec la priorité PRISIG (interruptible par des signaux).

```

int sys_nb_sigpend(){

    int nb = u.uarg[0];
    int cpt; long n ;

    struct proc *p= u.u_procp;

    while (1) {
        cpt =0 ;
        n = p->p_sig;

        for (i=1; i<NSIG ; i++){
            if (n & 1L)
                cpt++;
            n>>1;
        }

        if (n >= nb){
            u.uar0[R0] =n;
            return;
        }

        sleep (&(p->p_sig),PRISIG);
    } /while */
}

```

III.3. (1,5 points)

Si le processus appelant s'est endormi lors de l'appel à l'appel system *nb_sigpend (int nb)*, quelle fonction ira le réveiller? Justifier votre réponse.

La fonction `psignal( )` en appelant `setrun()`. De son côté, `psignal( )` est appelée par `kill( )`.

```

psignal (p,sig) {

    ...
    if (p-> p_stat == SSLEEP && p->p_pri > PZERO)
        setrun (p ) ;

}

```


ANNEXE

Code interne de wait()

```
1.  wait ()
2.  {
3.  register f, *bp;
4.  register struct proc *p;

5.  f=0;
6.  loop:
7.  for (p = &proc[0]; p < &proc [NPROC]; p++)
8.      if (p->p_ppid == u.u_procp -> p_pid) {
9.          f++;
10.         if (p->p_stat == SZOMB) {
11.             u.u_ar0[R0] = p->p_pid;
12.             bp = bread (swapdev, f =p->p_addr);
13.             mfree (swapmap, 1, f);
14.             p->p_stat = NULL;
15.             p->p_id = 0;
16.             p->p_ppid = 0;
17.             p->p_sig = 0;
18.             p_ttyp = 0;
19.             p->p_flag = 0;
20.             p = bp->b_addr;
21.             u.u_ar0[R1]= p->u_arg[0];
22.             brelse (bp);
23.             return;
24.         }
25.     }
26. if (f){
27.     sleep (u.u_procp, PWAIT);
28.     goto loop
29. }
30. u.u_error = ECHILD;
31. }
```